

# Department of Computer Science and Engineering

## Assignment Question

Patteela Yashwanth Kumar  
(192471055)

**Course Code & Name : CSA1206 – Computer Architecture**

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assignment Title:</b>       | <b>Solar Power Prediction System using Computer Architecture</b>                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Course Outcome(s) – CO:</b> | CO2:Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.<br>CO3:Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). |
| <b>Bloom's Taxonomy Level</b>  | L3 – Apply; L4 – Analyse                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>SDG Mapping (SDG 1-17)</b>  | <b>SDG 7 - Affordable and Clean Energy</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

**Problem Statement :** A solar farm collects solar irradiance, panel temperature, voltage, and current measurements to predict power generation. Design an arithmetic unit to calculate generated power using fixed-point and IEEE 754 floating-point arithmetic. Implement the computation using a pipelined datapath, identify possible hazards, and compare superscalar, out-of-order, and SMT approaches for processing data from multiple solar panels. Determine the architecture that provides high throughput with low energy consumption.

## Assessment Rubrics

| Criteria (CO Mapping)                                               | Max Marks | Excellent                                                                                                                                                                                                    | Good                                                                                                          | Satisfactory                                                                                                                           | Needs Improvement                                                                                |
|---------------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Sensor Data Representation &amp; Arithmetic Processing (CO2)</b> | 15        | Correctly represents soil-moisture, temperature, and humidity sensor values using suitable integer/floating-point formats and accurately applies arithmetic operations to determine irrigation requirements. | Correctly represents and processes most sensor values with minor errors in arithmetic or data representation. | Basic arithmetic processing is demonstrated, but some representation or calculation errors are present.                                | Sensor values are incorrectly represented or arithmetic processing is largely incorrect/missing. |
| <b>Application of Arithmetic Algorithms (CO2)</b>                   | 15        | Appropriately applies relevant integer/floating-point arithmetic concepts, including multiplication/division and IEEE 754 operations where required, with correct intermediate steps and results.            | Applies suitable arithmetic operations correctly with minor computational errors.                             | Uses basic arithmetic operations but demonstrates limited understanding of the underlying algorithms or floating-point representation. | Arithmetic algorithms are inappropriate, incorrectly applied, or absent.                         |
| <b>Pipelined Processor Datapath Design (CO3)</b>                    | 15        | Designs a clear and technically correct pipelined datapath for continuous sensor processing, with appropriate stages for sensor input, arithmetic computation, decision-making, and irrigation control.      | Datapath is mostly correct with minor issues in stage organization or data flow.                              | Basic pipeline stages are identified, but the datapath has incomplete organization or limited explanation.                             | Datapath is missing, poorly structured, or technically incorrect.                                |
| <b>Identification of Data Hazards &amp; Forwarding (CO3)</b>        | 10        | Correctly identifies data dependencies between sensor-processing instructions and demonstrates how forwarding resolves hazards without unnecessary pipeline stalls.                                          | Identifies most data hazards and applies forwarding appropriately with minor omissions.                       | Recognizes basic data hazards but forwarding implementation or explanation is incomplete.                                              | Data hazards are not correctly identified and forwarding is absent or inappropriate.             |

|                                                                                   |            |                                                                                                                                                                                                             |                                                                                                               |                                                                                                                        |                                                                                                            |
|-----------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Structural Hazards &amp; Stalling (CO3)</b>                                    | <b>10</b>  | Correctly identifies resource conflicts in the datapath and applies appropriate stalling/resource-management techniques with clear timing or pipeline-cycle analysis.                                       | Identifies major structural hazards and applies suitable stalling techniques with minor errors.               | Identifies basic resource conflicts but provides limited or partially correct hazard resolution.                       | Structural hazards are misunderstood or no suitable resolution technique is provided.                      |
| <b>Control Hazards &amp; Branch Prediction (CO3)</b>                              | <b>10</b>  | Correctly identifies control hazards caused by irrigation decisions/conditional branches and effectively applies branch prediction, explaining its impact on pipeline performance.                          | Correctly identifies control hazards and applies a suitable branch prediction approach with minor gaps.       | Basic understanding of control hazards is demonstrated, but branch prediction or its impact is inadequately explained. | Control hazards are not correctly identified and branch prediction is missing or inappropriate.            |
| <b>Pipeline Optimization, SMT Approaches &amp; Performance Evaluation (CO3)</b>   | <b>15</b>  | Quantitatively evaluates pipeline optimization using execution cycles, CPI, throughput, latency, speedup, and stall reduction; clearly demonstrates how optimization improves continuous sensor processing. | Performs most performance calculations correctly and explains the benefits of optimization with minor errors. | Provides basic performance comparison but lacks detailed calculations or interpretation.                               | Performance evaluation is missing, incorrect, or does not demonstrate the effect of pipeline optimization. |
| <b>Reflection: Design Decisions, Challenges &amp; Learning Outcomes (CO2/CO3)</b> | <b>10</b>  | Provides a thoughtful reflection on arithmetic choices, datapath design, hazard-resolution techniques, optimization decisions, challenges faced, and learning outcomes.                                     | Provides a clear reflection with reasonable discussion of design choices and challenges.                      | Reflection is present but generic and provides limited technical insight.                                              | Reflection is missing, superficial, or unrelated to the assignment.                                        |
| <b>Total</b>                                                                      | <b>100</b> |                                                                                                                                                                                                             |                                                                                                               |                                                                                                                        |                                                                                                            |

## Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

# Solar Power Prediction System Using Computer Architecture

## 1. Problem Understanding and Formulation

### 1.1 Problem Definition

A solar farm contains multiple photovoltaic (PV) panels whose power generation varies with solar irradiance, panel temperature, voltage, and current. The objective is to design a computer architecture that can process these sensor values efficiently and calculate the generated electrical power.

The basic power equation is:

$$P = V \times I$$

where:

- $P$  = generated power in watts (W)
- $V$  = panel voltage in volts (V)
- $I$  = panel current in amperes (A)

Solar irradiance and panel temperature are also considered because they influence the expected power output. The architecture therefore performs arithmetic processing, floating-point operations, pipelined execution, hazard handling, and parallel processing for multiple panels.

### 1.2 Expected Outcomes

The proposed system should:

1. Acquire solar-panel sensor data.
2. Represent sensor values using suitable fixed-point and floating-point formats.
3. Calculate power using multiplication.
4. Compare calculated power with expected/threshold power.
5. Process continuous sensor streams using a pipelined datapath.
6. Identify and resolve data, structural, and control hazards.
7. Compare superscalar, out-of-order (OoO), and simultaneous multithreading (SMT) approaches.
8. Achieve high throughput with low energy consumption.

### 1.3 Available Data

| Parameter         | Example Value        | Representation       |
|-------------------|----------------------|----------------------|
| Solar irradiance  | 850 W/m <sup>2</sup> | Fixed/Floating point |
| Panel temperature | 35 °C                | Fixed/Floating point |
| Voltage           | 40 V                 | Fixed/Floating point |
| Current           | 8 A                  | Fixed/Floating point |
| Power threshold   | 300 W                | Integer/Fixed point  |

For example:

$$P = 40 \times 8 = 320W$$

### **1.4 Assumptions**

- Sensor readings are available at regular sampling intervals.
- Voltage and current sensors provide calibrated values.
- Sensor noise is assumed to be within an acceptable range.
- The multiplication unit supports integer and floating-point arithmetic.
- A pipeline processes one or more sensor records continuously.
- Sensor communication and memory access are synchronized with the processor clock.
- The threshold value is predetermined according to the application.

### **1.5 Constraints**

- Low power consumption.
- High throughput for multiple panels.
- Minimum pipeline stalls.
- Accurate arithmetic calculations.
- Limited hardware resources.
- Correct handling of floating-point values.
- Real-time processing requirements.

## **2. Application of Course Knowledge**

### **2.1 Relevant Computer Architecture Principles**

The following concepts from Computer Architecture are applied:

- Integer arithmetic
- Fixed-point representation
- Floating-point arithmetic
- IEEE 754 standard
- Binary multiplication
- Pipelined datapath
- Data hazards
- Structural hazards
- Control hazards
- Forwarding
- Stalling
- Branch prediction
- Superscalar execution
- Out-of-order execution
- Simultaneous multithreading

### **2.2 Fixed-Point Representation**

Fixed-point arithmetic is useful when sensor values have a known range and precision.

For example, represent current with two decimal places using a scale factor of 100.

If:

$$I = 8.25A$$

then:

$$I_{fixed} = 8.25 \times 100 = 825$$

Similarly, if:

$$V = 40.50V$$

then:

$$V_{fixed} = 40.50 \times 100 = 4050$$

The actual power is:

$$P = \frac{4050 \times 825}{10000}$$

$$P = \frac{3,341,250}{10,000}$$

$$\boxed{P = 334.125W}$$

Fixed-point arithmetic reduces hardware complexity compared with floating-point arithmetic and can provide lower energy consumption.

### 2.3 Integer Multiplication

For integer sensor values, binary multiplication can be performed using conventional shift-and-add or Booth's multiplication algorithm.

For:

$$V = 40, I = 8$$

$$P = 40 \times 8 = 320$$

In a hardware implementation, Booth's algorithm reduces the number of addition/subtraction operations for signed binary operands and is suitable for a processor arithmetic unit.

### 2.4 IEEE 754 Floating-Point Representation

For measurements containing decimal values, IEEE 754 floating-point representation can be used.

A single-precision IEEE 754 number consists of:

| Field             | Bits |
|-------------------|------|
| Sign              | 1    |
| Exponent          | 8    |
| Fraction/Mantissa | 23   |
| Total             | 32   |

The value is represented as:

$$(-1)^S \times 1.F \times 2^{E-127}$$

Floating-point arithmetic is useful when sensor values have a wide range and greater numerical flexibility is required.

For example:

$$V = 40.5V$$

$$I = 8.25A$$

Therefore:

$$P = 40.5 \times 8.25$$

$$\boxed{P = 334.125W}$$

The floating-point multiplier performs the multiplication of the mantissas and adjusts the exponent according to IEEE 754 rules.

## 2.5 Solar Power Model

A simplified power model is:

$$P_{actual} = V \times I$$

An expected power value can be estimated using irradiance and temperature:

$$P_{expected} = P_{rated} \left( \frac{G}{G_{rated}} \right) [1 - \alpha(T - T_{ref})]$$

where:

- $P_{rated}$  = rated panel power
- $G$  = measured irradiance
- $G_{rated}$  = reference irradiance
- $T$  = panel temperature
- $T_{ref}$  = reference temperature
- $\alpha$  = temperature coefficient

Example

Assume:

$$P_{rated} = 400W$$

$$G = 850W/m^2$$

$$G_{rated} = 1000W/m^2$$

$$T = 35^\circ C$$

$$T_{ref} = 25^\circ C$$

and:

$$\alpha = 0.004/^\circ C$$

Then:

$$P_{expected} = 400(0.85)[1 - 0.004(35 - 25)]$$

$$= 340[1 - 0.04]$$

$$= 340(0.96)$$

$$\boxed{P_{expected} = 326.4W}$$

If measured voltage and current are:

$$V = 40.5V, I = 8.25A$$

then:

$$P_{actual} = 334.125W$$

Thus:

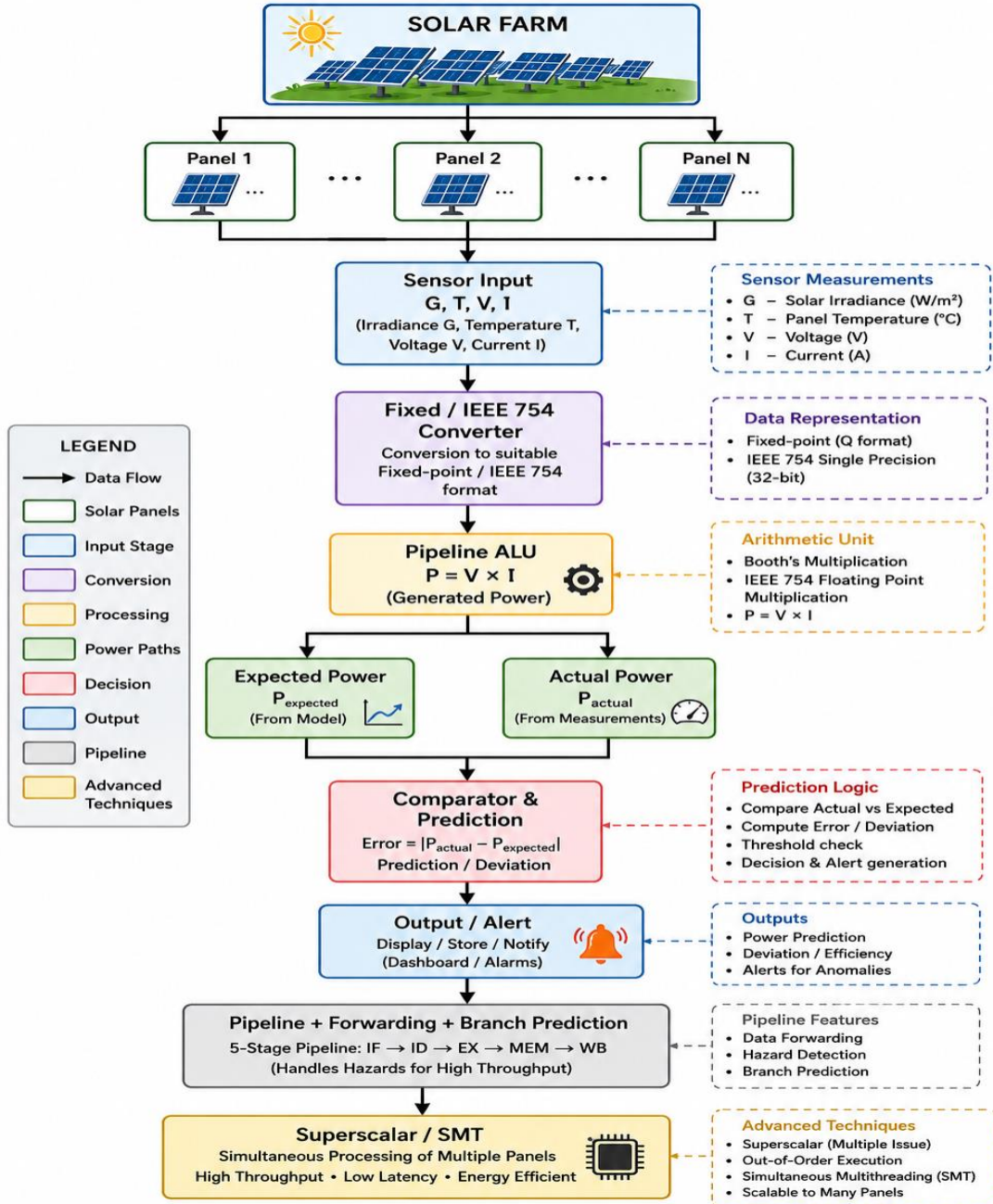
$$P_{actual} > P_{expected}$$

indicating that the panel is operating above the estimated power level.

### 3. Solution / Design / Methodology

#### 3.1 Proposed System Architecture

The proposed architecture contains the following stages:





### 3.2 Pipelined Datapath

A five-stage pipeline can be used.

| Stage | Operation                   |
|-------|-----------------------------|
| S1    | Sensor data acquisition     |
| S2    | Data conversion             |
| S3    | Arithmetic computation      |
| S4    | Power prediction/comparison |
| S5    | Output and control          |

Pipeline Representation

S1      S2      S3      S4      S5

Input → Conversion → Multiply → Compare → Output

For continuous sensor data:

Cycle    1    2    3    4    5    6

Panel 1   S1   S2   S3   S4   S5

Panel 2        S1   S2   S3   S4   S5

Panel 3                S1   S2   S3   S4   S5

Panel 4                        S1   S2   S3   S4

After the pipeline becomes full, one result can be produced every clock cycle, assuming no stalls.

### 3.3 Algorithm

Step 1

Read:

$$G, T, V, I$$

Step 2

Convert sensor values into fixed-point or IEEE 754 format.

Step 3

Calculate actual power:

$$P_{actual} = V \times I$$

Step 4

Calculate expected power:

$$P_{expected} = P_{rated} \left( \frac{G}{G_{rated}} \right) [1 - \alpha(T - T_{ref})]$$

Step 5

Compare:

$$P_{actual} \geq P_{expected}$$

Step 6

Generate the prediction/control result.

Pseudocode

START

Read G, T, V, I

Convert sensor data

$P_{\text{actual}} = V * I$

$P_{\text{expected}} = P_{\text{rated}} * (G / G_{\text{rated}}) * (1 - \alpha * (T - T_{\text{ref}}))$

IF  $P_{\text{actual}} \geq P_{\text{expected}}$   
    Status = NORMAL/HIGH

ELSE  
    Status = LOW

Send result to output

STOP

## 4. Use of Modern Tools

### 4.1 Python

Python can be used to:

- Generate solar sensor data.
- Perform fixed-point calculations.
- Perform floating-point calculations.
- Compare arithmetic errors.
- Calculate CPI and throughput.
- Plot power-generation graphs.

Example:

$V = 40.5$

$I = 8.25$

$P = V * I$

`print("Generated Power =", P, "W")`

Output:

Generated Power = 334.125 W

Python can also be used to simulate thousands of sensor readings and evaluate processor performance.

#### **4.2 MATLAB**

MATLAB can be used for:

- Solar power modelling.
- Sensor-data analysis.
- Graph generation.
- Floating-point error analysis.
- Pipeline performance analysis.

#### **4.3 Verilog/SystemVerilog**

A hardware description language can be used to model:

- Multiplication unit.
- Floating-point arithmetic unit.
- Pipeline registers.
- Forwarding unit.
- Hazard detection unit.
- Branch prediction logic.

#### **4.4 Simulation**

A processor simulator or HDL simulator can validate:

- Pipeline timing.
- Arithmetic results.
- Hazard handling.
- Number of stalls.
- Throughput and latency.

#### **Pseudocode**

START

FOR each solar panel

    Read irradiance G  
    Read temperature T  
    Read voltage V  
    Read current I

    Convert sensor values to fixed-point  
    OR IEEE 754 floating-point format

    Calculate actual power  
     $P_{\text{actual}} = V \times I$

```

Calculate expected power
    P_expected = Prated × (G / Grated)
                × [1 - α × (T - Tref)]

Calculate error
    Error = P_actual - P_expected

IF Error >= 0
    Status = "Above Expected"
ELSE
    Status = "Below Expected"
END IF

Store panel result

END FOR

Display power results
Generate performance graph

STOP

```

### **Implementation**

```

import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Panel": ["Panel 1", "Panel 2", "Panel 3", "Panel 4", "Panel 5"],
    "Irradiance": [850, 700, 950, 600, 1000],
    "Temperature": [35, 32, 40, 30, 45],
    "Voltage": [40.5, 38.0, 41.0, 35.0, 42.0],
    "Current": [8.25, 7.10, 8.80, 6.20, 9.00]
}

df = pd.DataFrame(data)

# Actual power calculation
df["Actual Power"] = df["Voltage"] * df["Current"]

```

```

# Expected power calculation
Prated = 400
Grated = 1000
alpha = 0.004
Tref = 25

df["Expected Power"] = (
    Prated * (df["Irradiance"] / Grated)
    * (1 - alpha * (df["Temperature"] - Tref))
)

df["Error"] = df["Actual Power"] - df["Expected Power"]

df["Status"] = df["Error"].apply(
    lambda x: "Above Expected" if x >= 0 else "Below Expected"
)

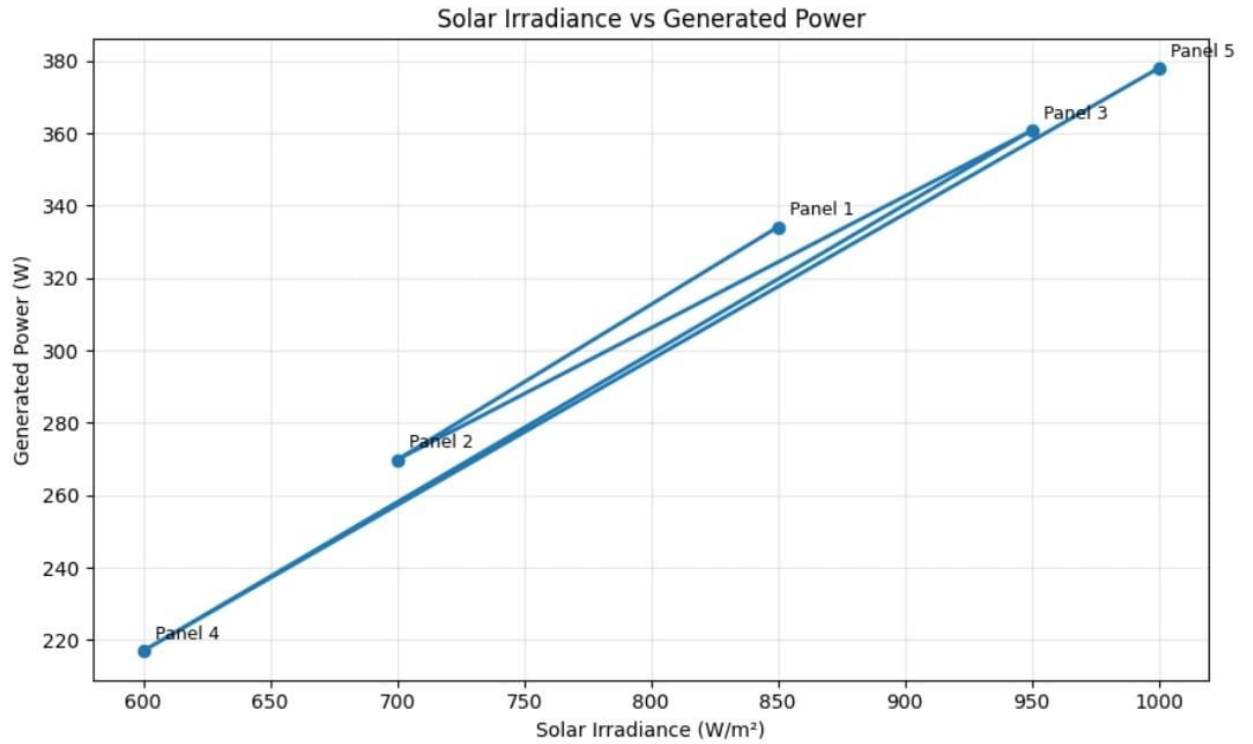
print(df.round(2))

# Graph
plt.figure(figsize=(9, 5))
plt.plot(
    df["Irradiance"],
    df["Actual Power"],
    marker="o",
    linewidth=2
)

for _, row in df.iterrows():
    plt.annotate(
        row["Panel"],
        (row["Irradiance"], row["Actual Power"]),
        xytext=(5, 5),
        textcoords="offset points"
    )

plt.title("Solar Irradiance vs Generated Power")
plt.xlabel("Solar Irradiance (W/m²)")
plt.ylabel("Generated Power (W)")
plt.grid(True, alpha=0.3)
plt.show()

```



## 5. Results and Validation

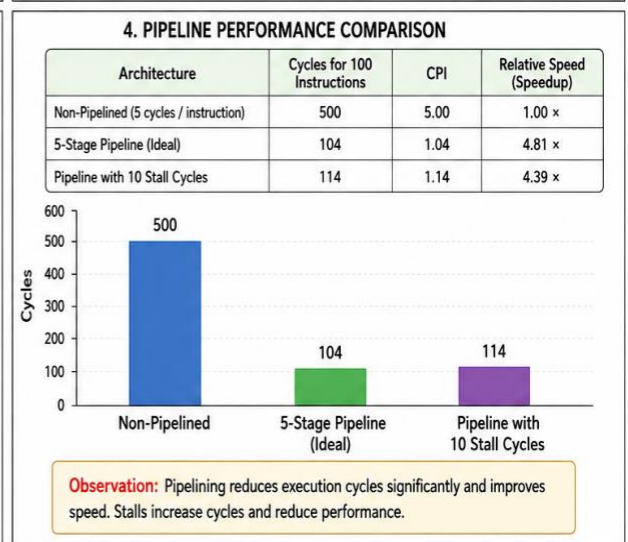
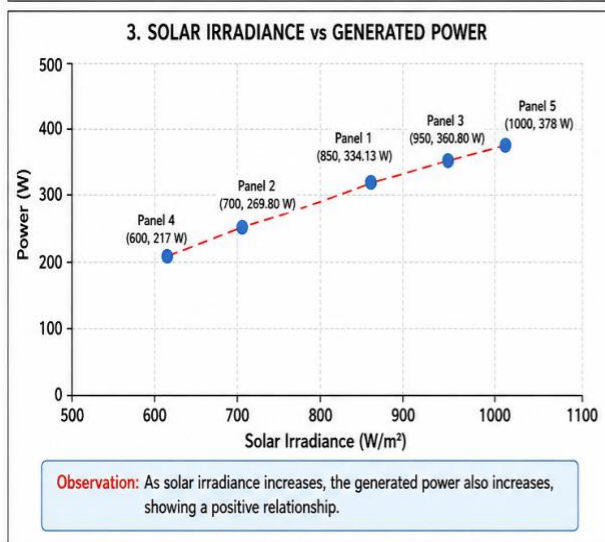
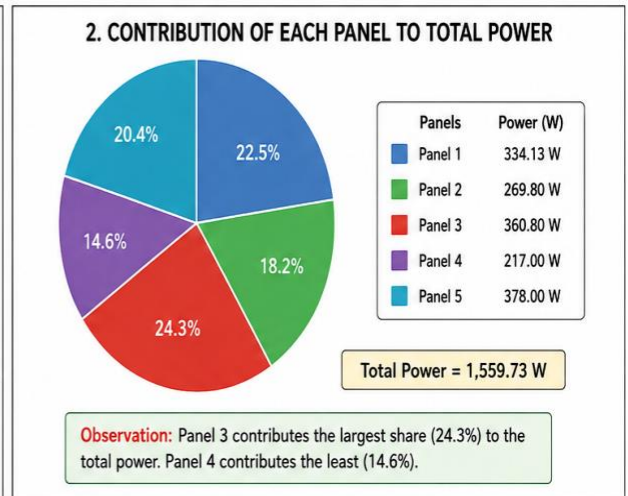
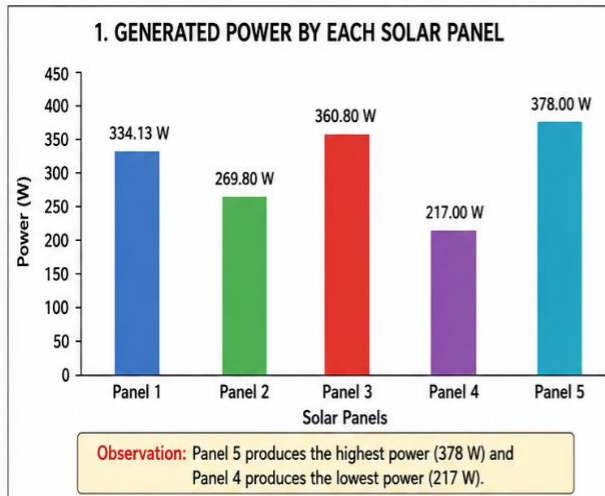
### 5.1 Sample Sensor Data

| Panel | Irradiance (W/m²) | Temp. (°C) | Voltage (V) | Current (A) | Actual Power (W) |
|-------|-------------------|------------|-------------|-------------|------------------|
| P1    | 850               | 35         | 40.5        | 8.25        | 334.13           |
| P2    | 700               | 32         | 38.0        | 7.10        | 269.80           |
| P3    | 950               | 40         | 41.0        | 8.80        | 360.80           |
| P4    | 600               | 30         | 35.0        | 6.20        | 217.00           |
| P5    | 1000              | 45         | 42.0        | 9.00        | 378.00           |

For Panel 1:

$$P = 40.5 \times 8.25 = 334.125W$$

Thus, the architecture correctly determines the generated power.



## 5.2 Fixed-Point vs Floating-Point

|                     |                  |                         |
|---------------------|------------------|-------------------------|
| Feature             | Fixed Point      | IEEE 754 Floating Point |
| Hardware complexity | Low              | High                    |
| Energy consumption  | Lower            | Higher                  |
| Range               | Limited          | Very wide               |
| Precision           | Predetermined    | Flexible                |
| Speed               | High             | Moderate                |
| Suitable for        | Embedded sensors | Complex calculations    |

Observation: Fixed-point arithmetic is preferable for simple, predictable sensor ranges, while IEEE 754 floating point is preferable when a wide numerical range and flexible precision are required.

## 6. Hazard Analysis

### 6.1 Data Hazard

A data hazard occurs when an instruction depends on the result of an earlier instruction.

Example:

I1:  $P = V \times I$

I2:  $R = P - P_{\text{expected}}$

I3: IF  $R > 0$

Here, I2 requires the result of I1.

Without Forwarding

I1: Multiply

I2: Stall

I3: Use P

With Forwarding

The result from the multiplication stage is directly forwarded to the next stage.

Multiply Unit

|  
| Forward P  
v

Comparison Unit

This reduces unnecessary pipeline stalls.

### 6.2 Structural Hazard

A structural hazard occurs when two operations require the same hardware resource at the same time.

For example:

Instruction 1  $\rightarrow$  Memory

Instruction 2  $\rightarrow$  Memory

If only one memory port is available, both cannot access memory simultaneously.

Solution

Use:

- Separate instruction and data caches.
- Multiple memory ports.
- Resource scheduling.
- Pipeline stalls when unavoidable.

### 6.3 Control Hazard

A control hazard occurs due to conditional decisions.

Example:

IF Pactual  $\geq$  Pexpected



HIGH/NORMAL  
 ELSE  
 LOW

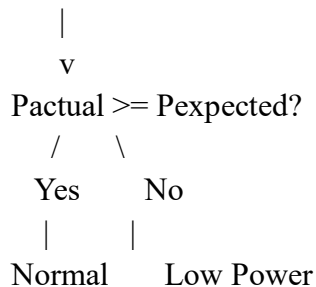
The processor may not know which instruction to fetch until the comparison result is available.

Branch Prediction

A branch predictor predicts the most likely result.

For example:

Branch prediction



Correct prediction reduces pipeline flushing and improves throughput.

## 7. Pipeline Performance Evaluation

Assume:

- Number of instructions = 100
- Pipeline stages = 5
- Ideal CPI = 1
- Clock cycle = 10 ns

Without Pipeline

If each instruction requires five cycles:

$$\text{Cycles} = 100 \times 5 = 500$$

Execution time:

$$500 \times 10\text{ns} = 5000\text{ns}$$

With Pipeline

For a 5-stage pipeline:

$$\begin{aligned} \text{Cycles} &= k + (n - 1) \\ &= 5 + (100 - 1) \\ &= \boxed{104 \text{ cycles}} \end{aligned}$$

Execution time:

$$104 \times 10\text{ns} = 1040\text{ns}$$

Speedup

$$\begin{aligned} \text{Speedup} &= \frac{500}{104} \\ &= \boxed{\text{Speedup} \approx 4.81} \end{aligned}$$

The ideal speedup approaches 5 for a five-stage pipeline.

### 7.1 Effect of Pipeline Stalls

Suppose there are 10 additional stall cycles.

$$Total\ cycles = 104 + 10 = 114$$

Therefore:

$$CPI = \frac{114}{100} = 1.14$$

Execution time:

$$114 \times 10ns = 1140ns$$

Forwarding and branch prediction can reduce these stalls and bring the CPI closer to 1.

## 8. Superscalar, Out-of-Order and SMT Comparison

| Feature                 | Superscalar                 | Out-of-Order                     | SMT                              |
|-------------------------|-----------------------------|----------------------------------|----------------------------------|
| Basic concept           | Multiple instructions/cycle | Execute ready instructions early | Multiple threads share processor |
| Parallelism             | Instruction-level           | Instruction-level                | Thread-level                     |
| Hardware complexity     | Medium                      | High                             | High                             |
| Throughput              | High                        | Very high                        | Very high                        |
| Energy                  | Moderate                    | High                             | Moderate-high                    |
| Suitable for solar farm | Good                        | Good                             | Excellent for many panels        |

### Superscalar

A superscalar processor can issue multiple instructions in one clock cycle.

For example:

Cycle 1:

Panel 1 → Multiply

Panel 2 → Multiply

This increases instruction throughput.

### Out-of-Order Execution

If one instruction is waiting for sensor data, another independent instruction can execute.

Instruction A → Waiting

Instruction B → Execute

Instruction C → Execute

This improves utilization but requires complex hardware such as reservation stations and reorder buffers.

#### SMT

SMT allows multiple threads to share processor resources.

For example:

Thread 1 → Panel 1

Thread 2 → Panel 2

Thread 3 → Panel 3

Thread 4 → Panel 4

This is particularly useful for a solar farm with many panels.

### **9. Analysis and Engineering Decision**

Three possible architectures were considered:

Option 1 – Simple Pipeline

Advantages:

- Low hardware complexity.
- Low energy consumption.
- Easy to implement.
- Good continuous throughput.

Limitation:

- Limited parallelism.

Option 2 – Superscalar + Pipeline

Advantages:

- Multiple instructions can execute simultaneously.
- Higher throughput.

Limitation:

- Higher hardware and energy requirements.

Option 3 – Pipeline + SMT

Advantages:

- Multiple solar-panel data streams can be processed concurrently.
- Good processor utilization.
- High throughput.
- Better resource sharing than using separate processors.

Limitation:

- More complex control logic.
- Shared resources can cause contention.

Final Decision

For a large solar farm, a pipelined superscalar processor with limited SMT support is a suitable architecture.

The pipeline provides continuous processing, while superscalar execution and SMT provide parallelism for multiple solar panels. Fixed-point arithmetic can be used for routine sensor calculations to reduce energy consumption, while IEEE 754 floating-point hardware can be selectively used for calculations requiring higher numerical flexibility.

Therefore, the recommended architecture is:

|                                                                     |
|---------------------------------------------------------------------|
| Pipelined + Superscalar + Selective SMT + Fixed/Floating Arithmetic |
|---------------------------------------------------------------------|

This provides a good balance between throughput, accuracy, hardware complexity, and energy consumption.

## **10. Broader Considerations**

### **10.1 Sustainability**

The system supports SDG 7 – Affordable and Clean Energy by improving solar-energy monitoring and utilization. Efficient processing can help identify underperforming panels and improve overall solar-farm operation.

### **10.2 Environmental Impact**

Solar power reduces dependence on fossil-fuel-based electricity. An energy-efficient processor further reduces the energy consumed by the monitoring system itself.

### **10.3 Economic Benefits**

Accurate monitoring can:

- Detect inefficient panels.
- Reduce maintenance time.
- Improve energy yield.
- Identify abnormal power generation.
- Reduce unnecessary replacement of healthy panels.

### **10.4 Safety**

Continuous monitoring can detect abnormal voltage, current, temperature, or power conditions and generate alerts before equipment damage occurs.

### **10.5 Accessibility and Professional Responsibility**

The system should provide understandable outputs and reliable measurements. Engineers must validate sensor calibration, protect collected data, and clearly communicate limitations of prediction models.

## **11. Conclusion**

The proposed Solar Power Prediction System using Computer Architecture processes solar irradiance, panel temperature, voltage, and current data to estimate and monitor solar power generation.

Fixed-point arithmetic provides an energy-efficient solution for sensor values with known ranges, while IEEE 754 floating-point arithmetic provides greater flexibility for complex calculations. A five-stage pipelined datapath consisting of sensor acquisition, conversion, arithmetic, comparison, and output stages provides high throughput.

Data hazards are reduced using forwarding, structural hazards are handled through resource management and stalling, and control hazards are reduced using branch prediction. Quantitative analysis shows that a five-stage pipeline can approach a speedup of 5 compared with non-pipelined execution under ideal conditions.

For large solar farms, combining pipelining, superscalar execution and SMT provides high throughput for multiple panel data streams. However, fixed-point arithmetic should be preferred wherever possible to reduce energy consumption.

The final architecture therefore provides an effective balance between computational performance, accuracy, scalability and energy efficiency, supporting reliable renewable-energy monitoring.

## **12. Student Reflection**

This assignment helped me understand how computer architecture concepts can be applied to a real-world renewable-energy application. I learned that arithmetic operations such as multiplication are not simply mathematical calculations; they require appropriate hardware and data representation in a processor.

I also understood the practical difference between fixed-point and IEEE 754 floating-point arithmetic. Fixed-point arithmetic can reduce hardware complexity and energy consumption, whereas floating-point arithmetic is useful when a wider range and greater flexibility are required.

The pipeline design helped me understand how multiple sensor readings can be processed simultaneously. Studying data, structural and control hazards showed me why forwarding, stalling and branch prediction are important for maintaining processor performance.

I also learned that choosing a processor architecture involves trade-offs. A highly parallel architecture can provide greater throughput but generally requires more hardware and energy. If additional resources were available, I would implement the complete design in Verilog/SystemVerilog, simulate multiple solar panels, generate performance graphs, and compare the energy consumption of fixed-point, floating-point, superscalar and SMT implementations.

## **13. References**

Use IEEE citation style for the final assignment.

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
  - [2] D. A. Patterson and J. L. Hennessy, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.
  - [3] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.
  - [4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019.
  - [5] Python Software Foundation, *Python Documentation*, Python Software Foundation.
  - [6] MathWorks, *MATLAB Documentation*, MathWorks.
  - [7] National Renewable Energy Laboratory (NREL), *Solar Resource and Solar Power Data*, U.S. Department of Energy.
- AI-Assisted Tool Acknowledgement
- [8] M. A. Green et al., “Solar cell efficiency tables (Version 57),” *Progress in Photovoltaics: Research and Applications*, vol. 29, no. 1, pp. 3–15, 2021.
  - [9] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, IEEE, 2020 edition/reference use.
  - [10] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2020/2022 reprints.